

Universidade Católica do Salvador (UCSal)

Projeto: Simulador de Batalhas Pokésal (Fase 02)

Data: 29 de setembro de 2026

4. Documento de Testes: Relatório Detalhado e Matriz de Rastreabilidade

Este documento detalha os testes automatizados desenvolvidos com o *framework* JUnit para validar as regras de negócio do simulador Pokésal, bem como a Matriz de Rastreabilidade exigida no edital para a Fase 02.

Relatório Detalhado dos Testes Automatizados (JUnit)

A arquitetura de testes foi focada na validação estrutural (valores limite) e comportamental (fluxo de exceções e aleatoriedade) do sistema. Foram implementadas sete classes de teste unitário:

1. **TestLimitetens.java:** Valida o lançamento da exceção `IllegalStateException`. O teste força o uso de três itens pelo mesmo Treinador. O JUnit assegura que a execução é interrompida na terceira tentativa, garantindo a integridade do limite de uso por batalha.
2. **TestOrdemDeAtaque.java:** Valida a lógica de turnos. Instancia dois objetos Pokesal com atributos de Velocidade (SPD) distintos e submete-os ao motor da classe Batalha. A asserção confirma que o fluxo de controle concede sempre a iniciativa ao objeto com maior SPD.
3. **TestVantagemElemental.java:** Avalia o multiplicador de matriz cruzada. O teste cruza ataques entre tipos elementais (ex: Fogo contra Planta, Água contra Fogo) e utiliza asserções matemáticas para garantir que o HP subtraído ao defensor reflete rigorosamente os multiplicadores de 2.0x (vantagem) e 0.5x (desvantagem).
4. **TestDanoBoundary.java:** Teste de valor limite (*Boundary Value Analysis*). Configura um cenário extremo onde a Defesa do alvo é matematicamente superior ao Ataque do agressor. O teste assegura que a trava estrutural do código é ativada, aplicando o dano mínimo de 1 e prevenindo a cura acidental do adversário.
5. **TestInstakill.java:** Valida o fator de aleatoriedade (RNG). Ao isolar a *seed* do objeto Random para simular os 2% de probabilidade, o teste confirma que o HP do alvo é reduzido a zero instantaneamente, ignorando os cálculos de defesa e multiplicadores.
6. **TestParalizado.java:** Verifica a interrupção do fluxo de estado. Ao forçar a flag `paralizado = true` num dos combatentes, o teste valida se o motor do turno salta a fase de `escolherAcao` desse objeto.
7. **TestEfeitoTerreno.java:** Avalia a injeção de efeitos passivos. O teste posiciona entidades no cenário "Canteiro Central" e valida se o incremento de 5% sobre o `hpMax` é aplicado estritamente aos monstros do tipo Planta no término do turno.

Matriz de Rastreabilidade de Testes

A tabela abaixo mapeia cada Regra de Negócio (Requisito) exigida no projeto à sua respectiva classe de teste automatizado, garantindo 100% de cobertura dos cenários críticos avaliados.

ID do Requisito	Descrição da Regra de Negócio / Funcionalidade	Classe de Teste JUnit Associada	Status de Cobertura
-----------------	--	---------------------------------	---------------------

RN-01	Limite máximo de uso de 2 itens por combate.	TestLimitItens.java	Validado
RN-02	O Pokésal com maior Velocidade (SPD) ataca primeiro.	TestOrdemDeAtaque.java	Validado
RN-03	Aplicação de multiplicadores de vantagem/desvantagem elemental (x2.0 e x0.5).	TestVantagemElemental.java	Validado
RN-04	Dano resultante menor que 1 deve ser convertido em 1 (dano mínimo garantido).	TestDanoBoundary.java	Validado
RN-05	Probabilidade de 2% de ocorrer um ataque fatal (<i>Instakill</i>).	TestInstakill.java	Validado
RN-06	Status "Paralisado" impede o alvo de executar o seu turno.	TestParalizado.java	Validado
RN-07	Bônus de regeneração de HP (5%) no final do turno para terreno específico.	TestEfeitoTerreno.java	Validado